

super-fast for large files with low memory usage. This gives a bytes-like object that can be terabytes in size but uses almost no RAM until you touch the pages, resulting in no RAM explosion.

DAWG (Directed Acyclic Word Graph): A DAWG is a highly compressed data structure used to store large sets of strings efficiently. It is especially powerful for dictionaries, NLP vocabularies, autocomplete engines, and prefix-based searches.

```
import dawg

words = ["cat", "car", "cart", "dog", "doing"]

d = dawg.CompletionDAWG(words)

print(d.keys("ca")) # prefix search: ['cat', 'car', 'cart']
print("dog" in d) # True
```

Tries: This refers to a tree-based data structure also known as a prefix tree or digital tree. It is used to efficiently store and retrieve a dynamic collection of strings by sharing common prefixes. Prefix-based string data structures play a crucial role in text indexing, NLP, search engines, autocomplete systems and large dictionary storage. Given below are a few commonly used structures.

Marisa-Trie (Matching Algorithm with Recursively Implemented Storage): This static compressed trie uses LOUDS (Level-Order Unary Degree Sequence) representation.

It has an extremely small memory footprint (often 5-10% of original text), and supports predictive search (common prefix search) and exact-match very efficiently. Once built, the structure is read-only, hence not suitable for frequent insertions/deletions. The Python module is marisa-trie.

Double-Array Trie (DAT): The Double-Array Trie (DAT) is a compact Trie representation that uses two arrays:

- a. BASE array determines offset for child transitions

```
base[i]: starting index for children of node i
```

- b. CHECK array stores the parent index to validate transitions

```
check[i]: parent node of node i (for collision detection)
```

It has very fast transitions (just array indexing) and supports dynamic insertion/deletion (though slower than lookup). The Python module is datrie.

HAT-Trie (Hash-Array mapped Trie): A HAT-Trie combines a trie for prefix organisation and hash buckets for

fast leaf-node lookups. It starts with a hash table at the top levels, then ‘bursts’ into small trie containers when buckets fill. It is extremely cache-efficient because burst containers are small and contiguous. It supports insertion, deletion, prefix search, and can store values/frequencies. The Python module is hat-trie

General tips

Avoid copies: Avoid making unnecessary copies of the objects, which will fill RAM with redundant data. Instead, use references or views from NumPy, or use iterators. Remember, for strings Python reuses identical immutable strings; this avoids storing the same string many times.

Profiling: Use the profiling methods to check memory usage while programming. These will help to detect early issues like excessive allocations, memory leaks, and inefficient structures.

Use generators instead of storing everything in memory: A generator yields one item at a time as we don’t need to store the entire dataset in RAM.

NumPy: While working with numeric data, use NumPy arrays as they offer many fast algorithms.

Bitarray: If your code requires lots of bit strings, both numpy and bitarray packages provide efficient representation of bits packed into bytes.

Micro Python: Good for working with embedded systems, its tiny memory footprint allows developers to write high-level, readable Python code while still running on hardware traditionally restricted to low-level C/C++. It brings much of the core Python 3 functionality to devices with extremely limited resources—often with as little as 16-256KB of RAM and flash storage.

Unit test suite: Even as you focus on optimisation do not forget the fundamentals — the purpose of the code. Make sure to have unit test suite in place before you make algorithmic changes.

Just to recap, profiling helps you see the problem; effective structures and techniques help you fix it. Together, they form a solid foundation for writing memory-efficient, high-performance Python programs. The general tips and practices help improve performance, reduce memory pressure, and make applications more scalable—especially when working with large datasets or in resource-constrained environments. **END** 🐧

 **By: Thangaselvi Arichandrapandian**

The author is a lifelong learner influenced by the quote:
“The more I learn, the more I realise how much I don’t know.”

— Albert Einstein.